NAME :- Lokare Mahir Abdul Majeed

ROLL NO :- 23AI34

BATCH :- 02

EXPERIMENT NO :- 05

AIM :-

To implement an autoencoder for image compression and reconstruction.

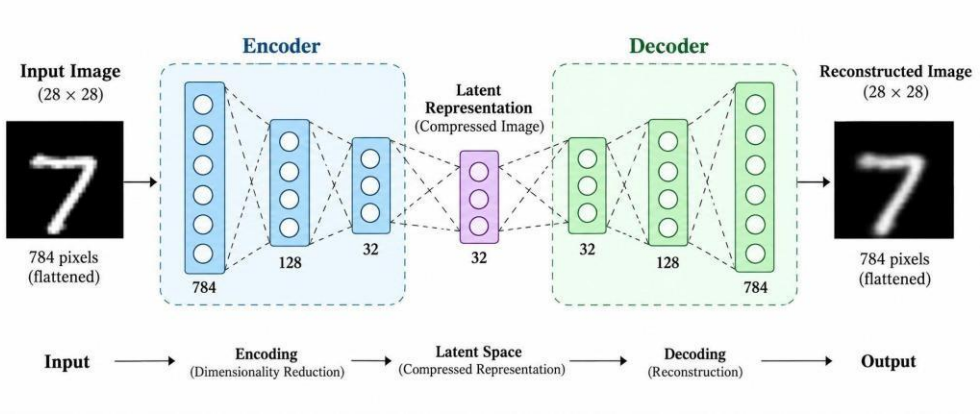
OBJECTIVE :-

1. To understand the working of an autoencoder.
2. To compress image data into a lower-dimensional representation.
3. To reconstruct the original image from the compressed representation.
4. To evaluate the quality of reconstructed images.

THEORY :-

Autoencoder :-

An Autoencoder is a neural network that learns to represent input data in a compressed form and then reconstruct the original input. It consists of three main components: Encoder, Latent Space, and Decoder.



The Encoder reduces the dimensionality of the input image and produces a compact representation called the latent representation. The Decoder uses this representation to reconstruct the original image.

Compression Process:

The input image is passed through the encoder, which extracts important features and stores them in a smaller latent vector. This latent vector represents the compressed form of the image.

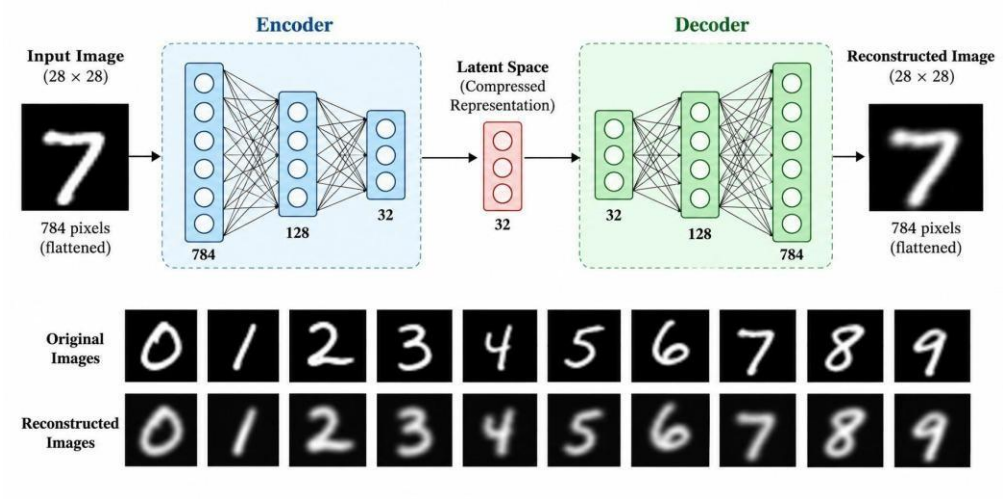
Reconstruction:

The decoder takes the compressed representation and attempts to reproduce an image as close as possible to the original input.

Loss Function:

Autoencoders commonly use reconstruction loss to measure the difference between the original and reconstructed images.

$$L = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2$$



PROCEDURE :-

1. Import TensorFlow and Matplotlib.
2. Load the MNIST dataset.
3. Normalize the image pixel values.
4. Flatten the 28×28 images into 784-dimensional vectors.
5. Design the Encoder using Dense layers.
6. Create a lower-dimensional latent representation.
7. Design the Decoder to reconstruct the input.
8. Compile the autoencoder using an optimizer and reconstruction loss.
9. Train the autoencoder using the input images as both input and target.
10. Reconstruct test images using the trained model.
11. Compare original and reconstructed images.
12. Plot the training loss.

CODE :-

```
import tensorflow as tf
import matplotlib.pyplot as plt

(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

input_img = tf.keras.Input(shape=(784,))

encoded = tf.keras.layers.Dense(
    128,
    activation="relu"
)(input_img)

encoded = tf.keras.layers.Dense(
```

```
    32,
    activation="relu"
)(encoded)

decoded = tf.keras.layers.Dense(
    128,
    activation="relu"
)(encoded)

decoded = tf.keras.layers.Dense(
    784,
    activation="sigmoid"
)(decoded)

autoencoder = tf.keras.Model(
    input_img,
    decoded
)

autoencoder.compile(
    optimizer="adam",
    loss="mse"
)

history = autoencoder.fit(
    x_train,
    x_train,
    epochs=10,
    batch_size=128,
    validation_split=0.2
)

reconstructed = autoencoder.predict(
    x_test[:10],
    verbose=0
)

test_loss = autoencoder.evaluate(
    x_test,
    x_test,
    verbose=0
)

print(
    "Test Reconstruction Loss:",
    test_loss
)

plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.title("Autoencoder Training Loss")

plt.legend()
plt.grid(True)

plt.show()

plt.figure(figsize=(12, 4))

for i in range(5):
```

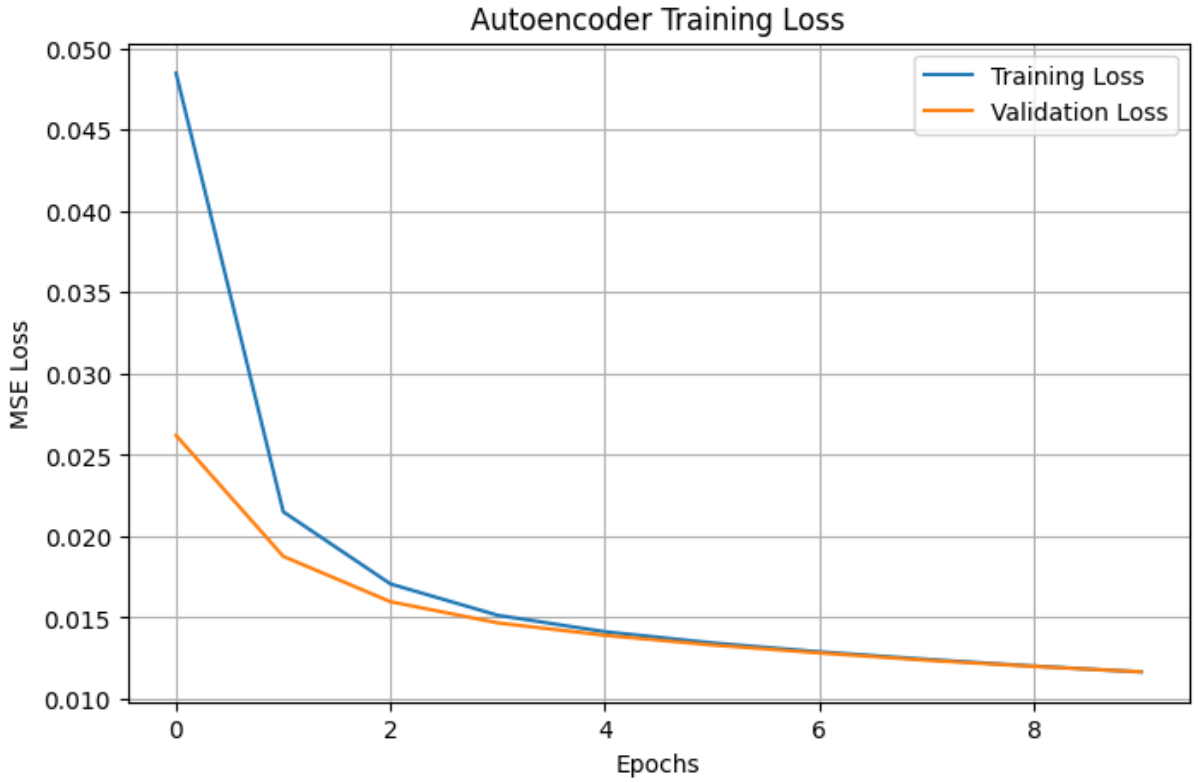
```
    plt.subplot(2, 5, i + 1)
    plt.imshow(
        x_test[i].reshape(28, 28),
        cmap="gray"
    )
    plt.axis("off")

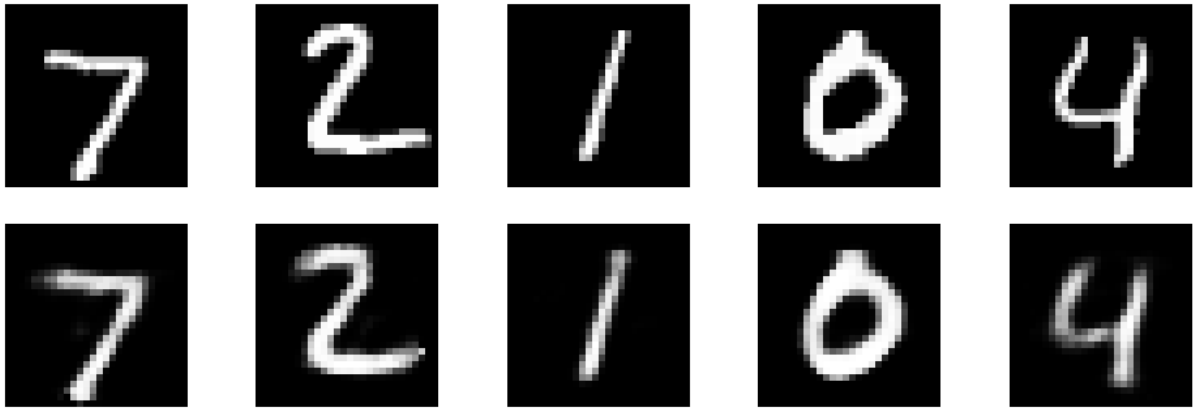
    plt.subplot(2, 5, i + 6)
    plt.imshow(
        reconstructed[i].reshape(28, 28),
        cmap="gray"
    )
    plt.axis("off")

plt.show()
```

OUTPUT :-

```
Epoch 1/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - loss: 0.0485 - val_loss: 0.0262
Epoch 2/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0215 - val_loss: 0.0188
Epoch 3/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0171 - val_loss: 0.0160
Epoch 4/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0151 - val_loss: 0.0147
Epoch 5/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0141 - val_loss: 0.0139
Epoch 6/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - loss: 0.0134 - val_loss: 0.0133
Epoch 7/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0129 - val_loss: 0.0128
Epoch 8/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - loss: 0.0124 - val_loss: 0.0124
Epoch 9/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0120 - val_loss: 0.0120
Epoch 10/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0117 - val_loss: 0.0117
Test Reconstruction Loss: 0.011229426600039005
```





CONCLUSION :-

The experiment successfully implemented an autoencoder for image compression. The model learned a compact latent representation and reconstructed the input images with low reconstruction loss.